Advanced Programming

Project assignment – 2nd stage
Factory method
Facade
Files and Serialization

Stage 2

- The student responsible for this task should start by creating a new *branch* from the `main` *branch*
  - *Branch* name: **`stage2`**

- The remaining students should update the project and then *checkout* the *branch*

Stage 2

- Features to develop in this stage
  - Chess game class
    - Represents and manages the chess game's operations
  - Restricting access to internal classes
    - Should use the *Facade* pattern
  - Centralized piece creation
    - Should use the *Factory Method* pattern
  - Data Persistence
    - Saving and restoring a game
      - Using object serialization
    - Importing and exporting game states
      - Reading and writing textual board representations

- **Students should refer to the theoretical class materials to complete the tasks planned for this stage**

Chess game class

- Should manage a chess game
- Must maintain a reference to an instance of an object representing the board
- Should support game creation:
  - Initially, it must allow the creation of a complete game
  - Later, as part of implementing data import and export functionalities, it should also support importing a partial game
- Should encapsulate the internal complexity of data-representing classes, acting as a *Facade* class

Chess game class

- Must provide methods to:
  - Query the board state
  - Execute a move
  - Retrieve the current player
  - Check if the game has ended and determine the winner
  - Other relevant operations
- Must not return or accept instances of internal objects
  - Should favor returning data and receiving parameters of primitive or immutable types (such as strings, enums, and records).

Piece Factory

- To simplify chess piece creation, two *Factory Methods* should be implemented:
  - A *Factory Method* that creates a piece based on its type (an enum), along with the required additional parameters
  - A *Factory Method* that creates a piece from its textual representation, following the previously explained format, along with additional parameters

Data model persistence

- Adapt all data classes to support serialization and deserialization

- Create a non-instantiable `ChessGameSerialization` class with two static methods to:
  - Serialize a `ChessGame` object
  - Deserialize a `ChessGame` object

Importing and Exporting partial games

- Add methods to the `ChessGame` class to support importing and exporting partial games
  - Partial games should be represented as a comma-separated string containing the text representation of each piece
  - The next player to move should also be included in this representation
  - Example:

```
BLACK,
Ra1*,Nb1,Bc1,Ke1*,Ng1,Rh1*,Pa2,Pb2,Pc2,Pd2,Pf2,Pg2,Ph2,
Qf3,Bc4,Pe4,qh4,pe5,pa7,pb7,pc7,pd7,pf7,pg7,ph7,ra8*,nb8,
bc8,ke8*,bf8,ng8,rh8*
```